20XT87 - DATA MINING LAB

GENOMEFOLIO — SYSTEM ARCHITECTURE

Team members:

22PT17 - Mohamed Aashir S, 21PT10 - Kiruthika V G

Overview

GenomeFolio is a small, layered Python package. Data flows in one direction and each stage is independently testable. The pipeline stops at the forecast.

```
prices -> returns -> symbol strings -> distance matrix -> clusters -> per-cluster forecast
```

Modules

- `src/config.py` — typed configuration. Loads `config/params.yaml` into frozen dataclasses and validates every value at startup.
- `src/data.py` — downloads adjusted close once, caches it, converts to daily returns, aligns to common trading days, and provides the committed fixture loader and a chronological split helper.
- `src/symbolize.py` — turns returns into the 5-symbol alphabet using per-stock (or pooled) quantiles, with a market-neutral option and explicit handling of NaNs and constant series.
- `src/align.py` — the graded substitution matrix, the banded affine-gap dynamic program (numba-compiled with a pure-Python fallback), a Smith-Waterman cross-check, and the normalised distance matrix.
- `src/cluster.py` — average-linkage clustering with constrained k selection, plus a band sweep and Adjusted Rand Index stability.
- `src/forecast.py` — per-cluster ARIMA (order by AIC, differencing by ADF) and a spline alternative, with a historical-mean fallback.
- `src/pipeline.py` — orchestrates the five stages for one window and returns clusters and forecasts.
- `src/cli.py` — the `genomefolio` console command.

Data flow

1. `load_fixture(cfg)` or `get_prices(cfg)` returns a dates-by-tickers price frame.
2. `to_returns` and `align_to_common_days` produce a clean returns frame.
3. The CLI takes the last `window.train_days` rows as the training window.
4. `run_pipeline` symbolizes the window, builds the distance matrix, clusters, aggregates each cluster into an equal-weight composite, and forecasts every composite.
5. The CLI prints the result and optionally writes JSON with `--out`.

Configuration schema

`config/params.yaml` has seven sections:

- `seed` — master random seed.
- `universe` — tickers, date range, cache directory, fixture path, minimum history.
- `split` — chronological tune/report fraction (available for holding out an evaluation period).
- `symbolize` — quantile cutoffs, ablation cutoffs, market-neutral flag, quantile scope.
- `align` — match score, gap open/extend, band, band sweep, numba flag.
- `cluster` — linkage, k range, k cap divisor, minimum cluster size.
- `window` — training-window length and forecast horizon.
- `forecast` — model, AIC grid bounds, seasonality flag, fallback policy.

Invalid values raise at load time so misconfiguration fails fast rather than mid-run.

Error handling

- Empty or partial configuration sections raise `KeyError`; out-of-range values raise `ValueError`.
- The alignment raises when the band is smaller than the length difference (the end cell would be unreachable).
- The clustering flags solutions that cannot satisfy the minimum cluster size instead of failing.

- The forecaster falls back to the historical mean and marks the result when no ARIMA model converges.

Tests

`tests/` mirrors the modules:

- `test_config.py` — defaults and validation.
- `test_data.py` — fixture size, return cleaning, chronological split.
- `test_symbolize.py` — alphabet range, quantile balance, constant and NaN handling.
- `test_align.py` — hand-worked alignment cases, symmetry, band guard, numba/pure-Python parity.
- `test_cluster.py` — block-structure recovery, k cap, minimum size, stability.
- `test_forecast.py` — valid orders, fallbacks, spline mode.
- `test_pipeline.py` — every stock clustered, every cluster forecast, determinism.

Extension points

- A new similarity measure only needs a function returning a symmetric, non-negative distance frame; `run_pipeline` can be pointed at it.
- A new forecast model plugs into `src/forecast.py` behind the existing `forecast_series` interface.
- A new symbolization scheme only needs to return an integer-coded frame with the same shape.

No-look-ahead guarantees

The pipeline receives a trailing window and never sees future rows. Symbol thresholds, alignment, clustering and the forecast are all computed from that window only. There is no global fitting step, so the design makes look-ahead difficult by construction.